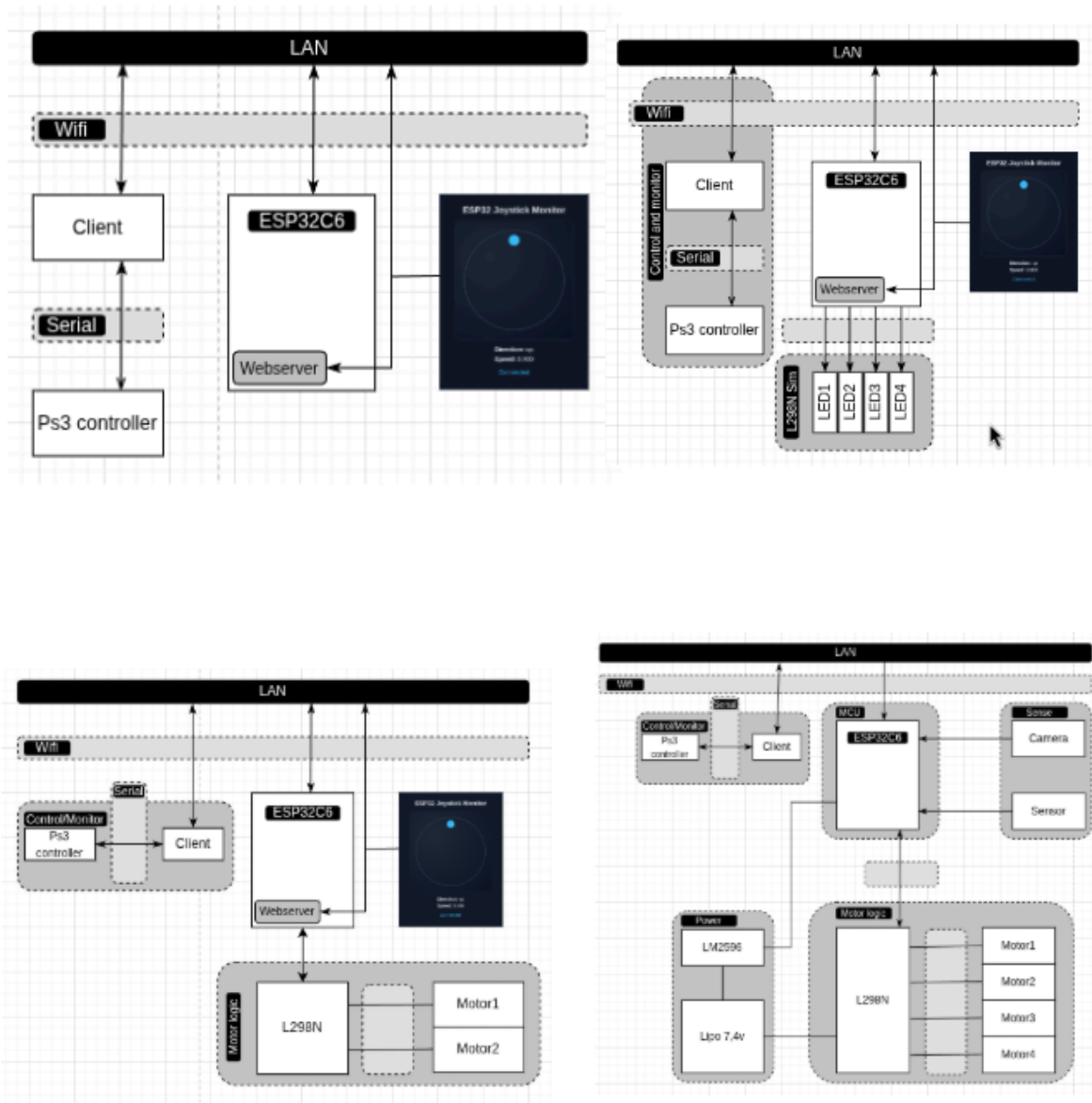


Kalibreringstest med PS3-controller

Efter studiet opstår der et vis tomrum, fordi der ikke er flere projekter og spænding tilbage, det meste af tiden gik med ansøgninger. Indimellem var der plads til at lave nogle spændende projekter og jeg ville styre en gammel robot jeg havde anvendt til AI eksamen, for at lave en iterativ proces, hvor jeg gradvist ville kunne styre en robot med en gammel PS3 controller, først fandt jeg ud af at PS3 kører på bluetooth classic og er ikke kompatibel med BLE på en ESP32C6, så jeg ændrede det til Wifi gennem en linux machine.



Det jeg nåede i processen, var at udvikle en test, hvor jeg fik nogle drivmotorer til at køre ved hjælp af en ESP32-C6. Ideen er dog, at systemet på sigt skal udvides med encoder-feedback og et kamera. Samtidig kan det blive nødvendigt at ændre

SoC-plattformen fra ESP32-C6 til eksempelvis en Raspberry Pi 5, hvis der skal køres computer vision.

Planen er først at træne og teste en YOLO-model, som derefter omsættes til bounding-box-detektion. Når modellen fungerer tilfredsstillende, vil den blive skaleret ned og optimeret til at køre på en Raspberry Pi 5. Formålet er at opnå stabile og brugbare frame rates, så systemet kan anvendes i praksis uden betydelig forsinkelse i billedbehandling.

ESP32-C6 vil i denne arkitektur kunne fungere som ROS-agent i systemet (MCU). Her vil den håndtere realtidsopgaver som styring af drivmotorer, sensorlogik samt encoder-feedback og sende denne information videre til den overordnede SoC. SoC'en – eksempelvis en Raspberry Pi 5 – vil i stedet håndtere de mere beregningstunge opgaver som kamera, brugerinput fra controller samt objekt-detektion.

Indtil videre har jeg testet og kalibreret ps3 joystik og opstillet en mere iterativ proces:

1. Opstilling af basal kalibreringslogik
2. Opstilling af logik, efter kalibrering med 4 LED som direction guides
3. Opstilling af test med motorer (Er pt. På dette trin)
4. Et skridt længere i processen, når jeg har testet hvor god en ESP32c6 er i forbindelse med kamera, hvis den viser sig at have problemer vil jeg skifte til en SoC

Alle tests foretaget i ESP-IDF

Test opstilling:



Jeg fandt i processen ud af at en PS3 controller ikke fungerer med nyere versioner af python og måtte backtracke til en tidligere version. Jeg havde endvidere forsøgt med en ESP32Wroom, da den har bluetooth classic og ikke BLE - Endvidere med en microDO, men ændrede det til at skulle køre over wifi, så PS3 controlleren kunne styre en ESP32C6 istedet.

Terminal feedback:

```

skradfy@skradfy:~/esp32_robot_ws$ ./pc_joystick_udp --ip 192.168.1.55 --dev /dev
/input/js0 --verbose
Streaming /dev/input/js0 -> 192.168.1.55:3333 (deadzone=0.10)
sent {"dir":"stop","speed":0.000}
sent {"dir":"stop","speed":0.000}
sent {"dir":"stop","speed":0.000}
sent {"dir":"up","speed":0.423}
sent {"dir":"up","speed":0.753}
sent {"dir":"up","speed":1.000}
sent {"dir":"up","speed":0.505}
sent {"dir":"stop","speed":0.000}
sent {"dir":"up","speed":0.309}
sent {"dir":"up","speed":0.536}
sent {"dir":"up","speed":0.814}
sent {"dir":"up","speed":1.000}
sent {"dir":"up","speed":0.969}
sent {"dir":"up","speed":0.412}
sent {"dir":"stop","speed":0.000}

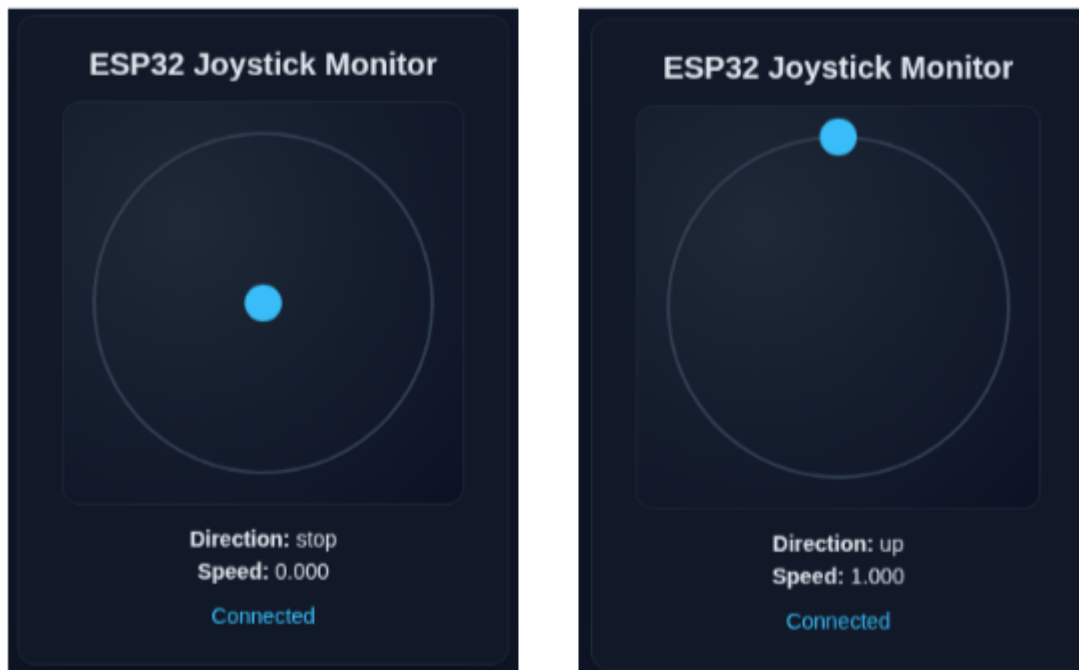
```

```

[joystick] dir=stop speed=0.000 raw={"dir":"stop","speed":0.000}
[wifi_cmd] rx len=28 payload={"dir":"stop","speed":0.000}
[joystick] dir=stop speed=0.000 raw={"dir":"stop","speed":0.000}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.423}
[joystick] dir=up speed=0.423 raw={"dir":"up","speed":0.423}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.753}
[joystick] dir=up speed=0.753 raw={"dir":"up","speed":0.753}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":1.000}
[joystick] dir=up speed=1.000 raw={"dir":"up","speed":1.000}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.505}
[joystick] dir=up speed=0.505 raw={"dir":"up","speed":0.505}
[wifi_cmd] rx len=28 payload={"dir":"stop","speed":0.000}
[joystick] dir=stop speed=0.000 raw={"dir":"stop","speed":0.000}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.309}
[joystick] dir=up speed=0.309 raw={"dir":"up","speed":0.309}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.536}
[joystick] dir=up speed=0.536 raw={"dir":"up","speed":0.536}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.814}
[joystick] dir=up speed=0.814 raw={"dir":"up","speed":0.814}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":1.000}
[joystick] dir=up speed=1.000 raw={"dir":"up","speed":1.000}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.969}
[joystick] dir=up speed=0.969 raw={"dir":"up","speed":0.969}
[wifi_cmd] rx len=26 payload={"dir":"up","speed":0.412}
[joystick] dir=up speed=0.412 raw={"dir":"up","speed":0.412}
[wifi_cmd] rx len=28 payload={"dir":"stop","speed":0.000}
[joystick] dir=stop speed=0.000 raw={"dir":"stop","speed":0.000}

```

Visual GUI feedback:



Første test code:

```
// ps3_calibrate.c
// Guided PS3 calibration: follow on-screen prompts (Up, Down, Left, Right),
// press the X / Cross button to advance to the next instruction. Logs axis
// events to CSV with the active step label.
// Build: gcc -O2 -Wall -Wextra -o ps3_calibrate ps3_calibrate.c
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <linux/joystick.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

#define AXIS_COUNT 12 // extend if your device exposes more axes
static const char *axis_names[AXIS_COUNT] = {
    "LX", "LY", "RX", "RY", "L2", "R2", "DPAD_X", "DPAD_Y",
    "AX8", "AX9", "AX10", "AX11"
};

// Button mapping: adjust if your X / Cross button has a different index.
// Common mappings: DS3 often maps Cross to 14, DS4 maps Cross to 1.
#define X_BUTTON_ID 1

static const char *steps[] = {
    "Press Up (Left Stick Forward)",
```

```

    "Press Down (Left Stick Backward)",
    "Press Left",
    "Press Right"
};

static const int STEP_COUNT = (int)(sizeof(steps) / sizeof(steps[0]));

static struct axis_stat {
    double min, max, last;
    int seen;
} stats[AXIS_COUNT];

static FILE *outf = NULL;
static int js_fd = -1;

static uint64_t now_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    return (uint64_t)ts.tv_sec * 1000ULL + ts.tv_nsec / 1000000ULL;
}

static void handle_sigint(int sig) {
    (void)sig;
    fprintf(stderr, "\n--- Axis ranges (norm) ---\n");
    for (int i = 0; i < AXIS_COUNT; i++) {
        if (!stats[i].seen) continue;
        fprintf(stderr, "%s: min %.3f max %.3f last %.3f\n",
            axis_names[i], stats[i].min, stats[i].max, stats[i].last);
    }
    if (outf) fclose(outf);
    if (js_fd >= 0) close(js_fd);
    exit(0);
}

int main(int argc, char **argv) {
    const char *dev_path = (argc >= 2) ? argv[1] : "/dev/input/js0";
    const char *csv_path = (argc >= 3) ? argv[2]
        :
"/home/skradfy/esp32_robot_ws/ps3_test_and_calibration/ps3_guided_calibration.csv";

    signal(SIGINT, handle_sigint);

    js_fd = open(dev_path, O_RDONLY);
    if (js_fd < 0) {
        perror("open js device");
        return 1;
    }

    outf = fopen(csv_path, "w");
    if (!outf) {
        perror("open csv");
        close(js_fd);
        return 1;
    }

    setvbuf(outf, NULL, _IOLBF, 0);
    fprintf(outf, "timestamp_ms,step,axis_name,axis_id,value_norm\n");

    int step_idx = 0;

```

```

printf("Starting guided calibration on %s\n", dev_path);
printf("Logging to %s\n", csv_path);
printf("Instruction: %s (press X to advance)\n", steps[step_idx]);

struct js_event e;
while (1) {
    ssize_t n = read(js_fd, &e, sizeof(e));
    if (n != sizeof(e)) {
        if (errno == EAGAIN || errno == EINTR) continue;
        perror("read");
        break;
    }

    // Kernel may tag init events; strip the init flag.
    if (e.type & JS_EVENT_INIT) e.type &= ~JS_EVENT_INIT;

    if (e.type == JS_EVENT_BUTTON && e.number == X_BUTTON_ID && e.value == 1) {
        step_idx++;
        if (step_idx >= STEP_COUNT) {
            printf("Completed all steps. Exiting...\n");
            break;
        } else {
            printf("Instruction: %s (press X to advance)\n", steps[step_idx]);
        }
    }

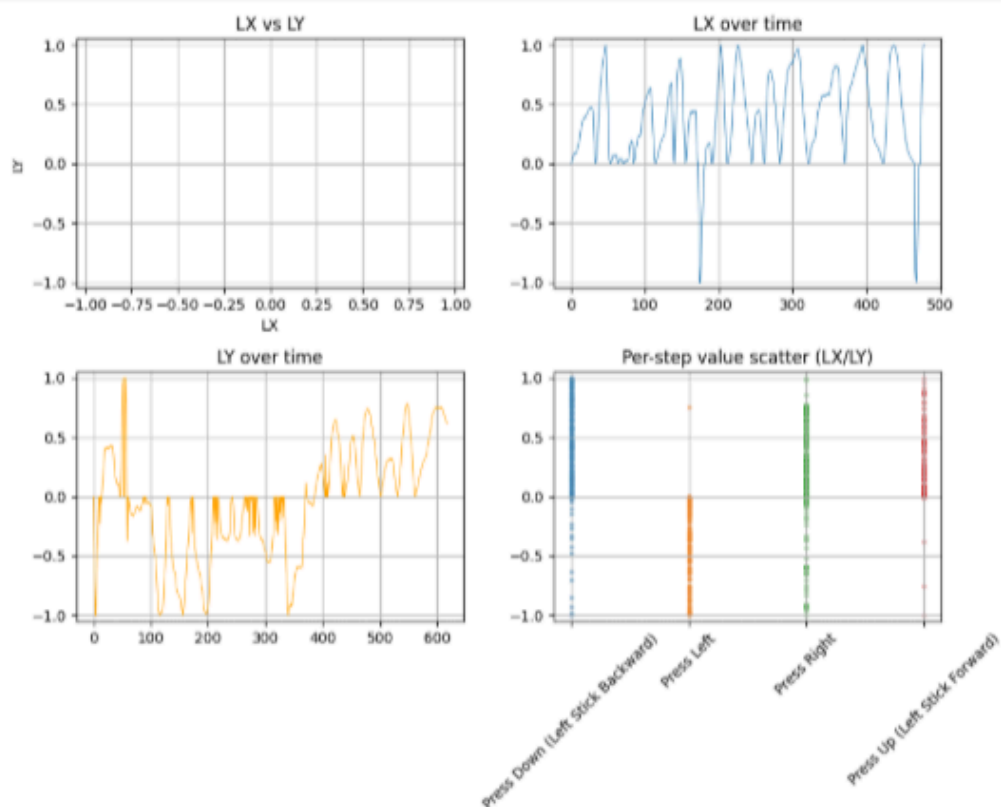
    if (e.type == JS_EVENT_AXIS && e.number < AXIS_COUNT) {
        double norm = (double)e.value / 32767.0; // range ~[-1, 1]
        uint64_t ts = now_ms();
        fprintf(outf, "%llu,%s,%s,%u,%.4f\n",
            (unsigned long long)ts,
            steps[step_idx],
            axis_names[e.number], e.number, norm);

        struct axis_stat *s = &stats[e.number];
        if (!s->seen) { s->min = s->max = norm; s->seen = 1; }
        if (norm < s->min) s->min = norm;
        if (norm > s->max) s->max = norm;
        s->last = norm;
    }
}

handle_sigint(0);
return 0;
}

```

Matlab feedback:



Endeligt mål er at have en robot, som jeg kan anvende til at teste forskellige ting som fx ROS2 eller mere realtime med en ESP32c6 som ROS Agent, robot billede 1 skal ombygges, den blev anvendt i arbejdet med AI presentation på 3. Semester, billede 2 er robotten under ombygning, når det jeg har en grund implementering så er planen at arbejde videre med min snegle robot, der også skal ombygges, da den er i tilstanden fra min 3. Semesters eksamen billede 3:

